

[НАИМЕНОВАНИЕ ОБРАЗОВАТЕЛЬНОЙ ОРГАНИЗАЦИИ]

[ФАКУЛЬТЕТ / ИНСТИТУТ]

[КАФЕДРА]

ОТЧЁТ ПО КУРСОВОМУ ПРОЕКТУ

**ИНФОРМАЦИОННАЯ СИСТЕМА ПОДДЕРЖКИ ПРИНЯТИЯ
РЕШЕНИЙ ПО ОПТИМИЗАЦИИ РАЗРАБОТКИ
КОМПЬЮТЕРНЫХ ИГР**

Выполнил: [ФИО студента]

Группа: [номер группы]

Руководитель: [ФИО, должность]

[ГОРОД]

2026

АННОТАЦИЯ

Отчёт посвящён разработке локальной информационной системы поддержки принятия решений для выбора технических способов реализации функций компьютерной игры. Система принимает профиль проекта, подбирает применимые методы, ранжирует их по нескольким критериям, показывает связь с инструментами игровых движков, выявляет ограничения и рассчитывает ориентировочный класс аппаратного обеспечения.

В работе зафиксирована граница предметной области: рассматриваются решения, влияющие на архитектуру, вычислительную нагрузку или потребление ресурсов. Монетизация, дата выхода, сюжетные развилки и количество концовок в расчёт не включаются. Сохраняются геймплейные решения, которые создают технические требования, в том числе разрушаемость окружения, масштаб мира, количество NPC и мультиплеер.

Основная функциональность проекта реализована. Текущая версия содержит каталог технических методов, многокритериальное ранжирование TOPSIS, расчёт ресурсов, корзину выбранных решений, объяснение результатов, административное наполнение и набор автоматических проверок. В документе оставлены поля для обновления после финальной очистки тестов, окончательной проверки быстрого старта и уточнения отдельных спорных записей каталога.

Ключевые слова: поддержка принятия решений, разработка компьютерных игр, техническая оптимизация, TOPSIS, аппаратная оценка, игровые движки, производительность.

СОДЕРЖАНИЕ

АННОТАЦИЯ	2
СОДЕРЖАНИЕ	3
СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ	5
ВВЕДЕНИЕ	6
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ	8
1.1 Особенности выбора технических решений	8
1.2 Пользовательские роли и потребности системы	8
1.3 Анализ существующих средств и границы их применимости	9
1.4 Границы предметной области	10
1.5 Постановка задачи и критерии качества	11
2 МОДЕЛЬ И МЕТОДЫ ПОДДЕРЖКИ ПРИНЯТИЯ РЕШЕНИЙ	12
2.1 Профиль проекта и технические входы	12
2.2 Паспорт технического метода	13
2.3 Фильтрация и оценка применимости	13
2.4 Ранжирование и устойчивость решения	14
2.5 Корзина и связи между решениями	15
2.6 Модель нагрузки и аппаратной оценки	15
2.7 Выводы по разделу 2	15
3 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ СИСТЕМЫ	16
3.1 Общая архитектура	16
3.2 Предметные данные и публикация	17
3.3 API и единый результат расчёта	18
3.4 Состояние frontend и сохранение текущего проекта	18
3.5 Запуск на Windows и Linux	19
3.6 Выводы по разделу 3	19
4 ТЕХНИЧЕСКИЕ МЕТОДЫ И ИХ ВЛИЯНИЕ НА ИГРУ	20
4.1 Механизмы, эффекты и ограничения методов	20
4.1.1 Управление видимостью и детализацией	20
4.1.2 Batching и инстансинг	21
4.1.3 Поточковая загрузка, виртуальное текстурирование и сжатие	21
4.1.4 Предвычисление, кэширование и расписание	21
4.1.5 Трассировка лучей, материалы и постобработка	22

4.2 Геймплейные решения как технические требования	23
4.3 Интеграция, совместимость и стоимость замены	23
5 МАТЕМАТИЧЕСКИЕ МОДЕЛИ, ПРОВЕРКА И РЕЗУЛЬТАТЫ	24
5.1 Модель ресурсной нагрузки	24
5.2 Ранжирование методом TOPSIS	24
5.3 Организация проверки	25
5.4 Чувствительность к изменению параметров профиля	25
5.5 Изменение и сохранение корзины	26
5.6 Ограничения достоверности и аппаратной оценки	27
5.7 Текущее состояние тестов и сборки	28
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ А. СВОДКА ТЕКУЩЕГО СОСТОЯНИЯ ПРОЕКТА	31
ПРИЛОЖЕНИЕ Б. КОМПАКТНЫЙ ПРИЁМОЧНЫЙ ПРОТОКОЛ	32
ПРИЛОЖЕНИЕ В. ПОЛЯ ДЛЯ ФИНАЛЬНОГО ОБНОВЛЕНИЯ	33

СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ

Таблица 0.1 - Основные сокращения

Сокращение	Расшифровка	Использование в работе
API	Application Programming Interface	программный интерфейс backend
CPU	Central Processing Unit	центральный процессор
GPU	Graphics Processing Unit	графический процессор
RAM	оперативная память	память системы
VRAM	видеопамять	память графического устройства
RT	Ray Tracing	трассировка лучей
LOD / HLOD	Level of Detail / Hierarchical LOD	уровни детализации
NPC	Non-Player Character	неигровой персонаж
TOPSIS	Technique for Order Preference by Similarity to Ideal Solution	многокритериальное ранжирование

ВВЕДЕНИЕ

Производительность компьютерной игры определяется не одной настройкой, а взаимодействием нескольких подсистем: подготовки команд рендера, растеризации, освещения, физики, искусственного интеллекта, анимации, потоковой загрузки, памяти и сетевой синхронизации. Поэтому решение, принятое на стадии проектирования, может повлиять на структуру данных, набор инструментов движка, порядок подготовки контента и требования к оборудованию. Задача системы состоит в том, чтобы сделать такие зависимости видимыми до появления готового игрового билда.

Актуальность проекта определяется тем, что существующие средства обычно решают отдельные задачи. Документация движка описывает конкретный механизм, профилировщик измеряет уже созданный фрагмент игры, а каталог оборудования сообщает характеристики устройств. Между этими источниками остаётся практическая задача выбора: какой способ реализации соответствует функциям и ограничениям конкретного проекта, какие компромиссы он создаёт и что произойдёт при замене решения в уже сформированном наборе.

Цель проекта - разработать информационную систему поддержки принятия решений, которая связывает профиль игры, технические функции, варианты их реализации, совместимость решений и ориентировочные требования к ресурсам. Система должна выдавать объяснимый результат и явно отделять расчётную модель от измеренных данных.

Объект исследования - технические решения, принимаемые при проектировании и разработке компьютерных игр. Предмет исследования - модели представления таких решений, правила их применимости и алгоритмы ранжирования с учётом влияния на архитектуру, вычислительную нагрузку и ресурсы.

Для достижения цели решены следующие задачи:

- определена предметная область и выделены входные параметры профиля проекта;
- сформирован каталог игровых функций, технических методов, движков, инструментов и аппаратных записей;

- реализованы фильтрация недопустимых решений и многокритериальное ранжирование применимых вариантов;
- предусмотрены объяснения результата, условия применения, конфликты, зависимости и взаимодополняющие связи;
- разработана ориентировочная модель стоимости кадра, памяти и подбора референсного класса оборудования;
- реализована корзина текущих решений с пересчётом результата после изменения профиля или состава набора;
- проведены проверки чувствительности к входным параметрам, ограничений, сохранения и защиты от устаревших результатов.

Методы работы включают декомпозицию игровой нагрузки по подсистемам, формализацию правил предметной области, многокритериальный метод TOPSIS, анализ связей между решениями, тестирование граничных и сценарных случаев, а также проверку воспроизводимости расчёта. Коэффициенты нагрузки в текущей реализации являются экспертными оценками; они не заменяют профилирование конкретной игры.

Практическая значимость заключается в том, что система позволяет сформировать обоснованный предварительный план технических решений до интеграции их в игровой проект. Результат используется как средство анализа и подготовки прототипа, а не как автоматическое доказательство достижения заданного FPS.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Особенности выбора технических решений

В разработке игры одна и та же функция может быть реализована несколькими способами. Например, потоковая загрузка может опираться на разбиение мира, кэширование страниц или виртуальное текстурирование; управление детализацией может использовать отдельные уровни мешей, иерархические группы или импосторы. Способ реализации меняет не только локальную стоимость кадра, но и требования к подготовке контента, памяти, загрузке, инструментам движка и проверке качества.

Важным является различие между функцией игры и методом её реализации. Функция отвечает на вопрос, что должно работать: открытый мир, физика разрушения, толпа NPC или сетевое взаимодействие. Метод отвечает на вопрос, как это организовать: culling, LOD, batching, стриминг, ограничение частоты обновления, репликация или другой механизм. Такая пара позволяет связать пользовательское требование с инженерным решением.

Для анализа стадий разработки использованы идеи разделения проектных параметров, измеряемых характеристик и регрессионных зависимостей, представленные в исследовании GAMORRA [1]. Коэффициенты из внешней работы в систему не переносятся: различаются входные параметры, наборы игр и аппаратные условия. Источник используется как основание для разделения модели и измерительного протокола.

1.2 Пользовательские роли и потребности системы

Пользователь системы - команда или разработчик, которому нужно принять техническое решение на стадии концепции, прототипа или производства. Роли ниже являются функциональными: в небольшой команде их может совмещать один человек.

Таблица 1.1 - Роли пользователей системы

Роль	Что требуется решить	Результат работы
Технический руководитель	выбрать архитектурный вариант и ограничения	приоритетный набор методов и аппаратный ориентир
Программист или technical artist	сопоставить метод с движком, контентом и пайплайном	условия применения, риски качества и интеграции
Аналитик проекта	проверить профиль, корзину и объяснимость результата	воспроизводимый расчёт и список вопросов для проверки

Ключевые требования к системе сформулированы в проверяемом виде:

- FR-01: принять профиль проекта, нормализовать его и явно обработать неизвестные значения;
- FR-02: отфильтровать неприменимые методы с объяснением причины и ранжировать допустимые варианты;
- FR-03: рассчитать текущую корзину, конфликты, зависимости и сводное влияние ресурсов;
- FR-04: подобрать референсный класс оборудования с учётом производительности, памяти и обязательных возможностей;
- NFR-01: для одного профиля и одной корзины выдавать воспроизводимый отпечаток входа и не показывать результат старого состояния.

Критерии качества дополняют требования: результат должен быть объяснимым, изменения значимых входов должны отражаться в расчёте, а экспертные оценки и модельные примеры должны быть отделены от измеренных данных.

Таким образом, потребности пользователя покрываются анкетой профиля, фильтрацией и ранжированием каталога, карточкой последствий, корзиной, аппаратной оценкой и сбросом устаревшего результата. Эти функции образуют единый проверяемый сценарий.

1.3 Анализ существующих средств и границы их применимости

Документация игровых движков является необходимым источником технических сведений, но обычно описывает отдельный инструмент и его ограничения. Она не выполняет сопоставление методов разных движков по критериям проекта, не формирует текущую корзину и не рассчитывает общий профиль ресурсов.

Профилировщики и средства захвата времени кадра полезны после появления работающей сцены. Они позволяют измерять CPU-, GPU- и display-времена, но требуют готового билда или инструментированного прототипа. В рамках проекта такие средства рассматриваются как внешняя проверка выбранного решения, а не как встроенная часть DSS. Это принципиально ограничивает область обещаний системы.

Каталоги оборудования и минимальные требования игр дают ориентир по устройствам, но не объясняют, какие подсистемы создают нагрузку и как меняется результат при замене метода. Кроме того, официальное требование часто относится к конкретному пресету, разрешению и версии игры. Поэтому в проекте аппаратный каталог используется для подбора референсного класса, а не для выдачи гарантии производительности.

Обобщённые системы архитектурного моделирования описывают потоки работ и ресурсы, но для текущей задачи нужна компактная модель с игровыми функциями, методами и условиями совместимости. Универсальное хранилище всех доказательств в границы проекта не входит.

1.4 Границы предметной области

Таблица 1.2 - Правило включения решений

Категория	Решение	Обоснование
Включается	разрушаемость окружения	влияет на физику, динамическую геометрию, кэширование и сеть
Включается	масштаб мира	влияет на стриминг, память, подготовку контента и резидентность
Включается	количество NPC	влияет на AI, анимацию, симуляцию и сетевую репликацию
Включается	мультиплеер	задаёт сетевую модель, частоты, роли сервера и требования к детерминизму
Включается	разрешение, FPS, качество	меняют бюджет кадра и стоимость соответствующих проходов
Исключается	монетизация	не является параметром вычислительной архитектуры в данной задаче
Исключается	дата выхода	не задаёт физическую стоимость выполнения игры
Исключается	сюжетные развилки и количество концовок	не создают сами по себе проверяемую ресурсную нагрузку

Критерий включения сформулирован как влияние на архитектуру, вычислительную нагрузку или потребление ресурсов. Это не утверждение о незначимости исключённых решений для игры в целом; они просто находятся за пределами рассматриваемой технической модели.

1.5 Постановка задачи и критерии качества

Система должна принимать профиль игры и возвращать набор применимых технических методов, ранжированный по выбранному приоритету. Для каждой записи требуется отображать причины выбора, условия применения, предполагаемые изменения ресурсов, влияние на качество и стадию внедрения. Неприменимые записи не должны исчезать бесследно: причина исключения должна быть видна пользователю.

Критерии качества: детерминированность, объяснимость, согласованность экранов, отсутствие неподтверждённых обещаний и реакция на значимый вход. После изменения профиля или корзины старый результат не должен оставаться видимым для нового состояния.

Основная функциональность по этим критериям реализована. Финальная чистка тестов, быстрый старт и дополнительная проверка каталога вынесены в приложение В.

2 МОДЕЛЬ И МЕТОДЫ ПОДДЕРЖКИ ПРИНЯТИЯ РЕШЕНИЙ

2.1 Профиль проекта и технические входы

Профиль ProjectProfile содержит 41 поле. Поля сгруппированы по смыслу, чтобы не смешивать идентификацию проекта, требования геймплея, параметры активной сцены, ограничения ресурсов и политику выбора. Наличие поля в анкете не означает, что оно является измеренным параметром: для неизвестного значения система должна показывать допущение или снижать уверенность.

Таблица 2.1 - Группы входных параметров профиля

Группа	Примеры полей	Роль в расчёте
Контекст	format, world_type, scale, stage, engine, engine_version, platforms	применимость, платформенные и стадийные условия
Активная сцена	object_count_level, object_count, npc_count_level, npc_count, local_view_count	работа на кадр и оценка уверенности
Геймплейные требования	functions, multiplayer, player_count, simulation_radius_m	выбор подсистем и архитектурных ограничений
Рендеринг	target_resolution, target_quality, target_fps, render_api	бюджет кадра и стоимость пиксельных стадий
Симуляция	physics_tick_hz, audio_complexity, network_topology	отдельные частоты и виды нагрузки
Память и данные	memory_model, streaming_pool_gb, storage_type, size_limit_gb	состав памяти, потоковая загрузка и ограничения
Политика выбора	complexity_tolerance, deadline_weeks, priority, cpu_budget	фильтры, веса и риск проекта

Некоторые поля требуют аккуратного определения. Количество объектов не должно автоматически трактоваться как количество видимых объектов, а количество NPC - как число одновременно симулируемых агентов. Число игроков не равно числу камер и не задаёт топологию сети. Стадия разработки влияет на своевременность внедрения и стоимость поздней переработки, но не должна менять физическую стоимость одного и того же кадра без изменения реализации.

2.2 Паспорт технического метода

Запись каталога описывает не только название оптимизации. В ней фиксируются функция или общий уровень применения, вид решения, область эффекта, ограничения формата и движка, ресурсные эффекты, влияние на качество и концепцию, стоимость внедрения, стоимость позднего внедрения, рекомендуемая стадия, шаги реализации, примеры использования и источник.

Основная причинная цепочка в паспорте имеет вид: игровая функция -> базовый способ реализации -> заменяемая или сокращаемая работа -> технический метод -> затронутые ресурсы и проверяемые последствия. Если промежуточный механизм не определён, утверждение о причинном эффекте заменяется на краткое допущение или оставляется пустым.

В паспорте различаются четыре статуса: механизм, подтверждённый источником; экспертное допущение для численного коэффициента; измеренный результат конкретного стенда; открытый вопрос, для которого поле оставляется пустым или сопровождается короткой оговоркой.

2.3 Фильтрация и оценка применимости

Сначала система формирует кандидатов по выбранным функциям и добавляет общие методы. Затем для каждого метода выполняется проверка применимости. Жёсткими основаниями исключения являются несовместимый формат, тип мира, платформа, движок, отсутствующая возможность или превышение заданного ограничения. Мягкие факторы - стадия, допустимая сложность, срок и соответствие приоритету - влияют на оценку, риск и порядок, но не должны маскироваться под физическую невозможность.

Пошагово расчёт выполняется следующим образом:

- нормализуются входы профиля и удаляются дубликаты функций и платформ;
- из каталога выбираются опубликованные методы с указанным источником;
- применяются фильтры совместимости и формируются объяснения исключений;
- для допустимых методов формируется матрица критериев;

- матрица ранжируется по TOPSIS с весами, зависящими от приоритета пользователя;
- к результату добавляются риски стадии, связи с движком, альтернативы и устойчивость ранга;
- для текущей корзины отдельно рассчитываются выбранные методы, конфликты, зависимости и сводный профиль нагрузки.

Листинг 2.1 - Упрощённый алгоритм формирования результата

```
profile = normalize(profile_input)
basket = canonicalize(selected_methods)
input_key = hash(profile, basket)
if input_key != current_result_key:
    discard_result()
candidates = filter_catalog(profile)
ranked = topsis(candidates, profile.priority)
result = build_result(profile, ranked, basket, input_key)
return result
```

Листинг показывает логику взаимодействия компонентов, а не полный исходный код. В реальной реализации те же проверки разделены между backend-расчётом и состоянием frontend. После изменения профиля или корзины ответ старого запроса не должен возвращаться на экран.

2.4 Ранжирование и устойчивость решения

TOPSIS применяется к матрице «альтернатива - критерий». Критерии делятся на выгодные и затратные, после чего для каждой допустимой альтернативы рассчитывается близость к идеальному решению. Формулы нормализации и коэффициента близости приведены в разделе 5, чтобы математический аппарат был собран в одном месте.

В текущем профиле учитываются производительность, сохранение качества, соответствие концепции, стоимость внедрения, штраф позднего внедрения, риск, соответствие ресурсам и уверенность. При одной альтернативе или одинаковых строках матрицы строгий порядок не имеет смысла. Разница менее 0,02 трактуется как группа практически равнозначных вариантов, а не как доказанное превосходство лидера.

Для проверки устойчивости веса каждого критерия по одному изменяются на $\pm 10\%$. Система показывает диапазон мест, которое может занимать метод. Такая процедура показывает чувствительность результата к политике выбора, но не превращает экспертные исходные оценки в

измеренные данные.

2.5 Корзина и связи между решениями

Корзина хранит один текущий набор решений и профиль проекта. Для набора проверяются конфликты, зависимости и взаимодополняющие связи. Конфликт означает, что решения нельзя одновременно применять в заявленной конфигурации. Зависимость означает наличие предварительного условия. Связь complement сама по себе не является измеренным численным бонусом: дополнительный эффект разрешено добавлять только при наличии обоснования совместного механизма.

Состав корзины нормализуется: дубликаты удаляются, порядок кодов не влияет на отпечаток входа. После изменения профиля или корзины старый результат сбрасывается, а ответ устаревшего запроса отбрасывается.

Сложность замены нельзя вывести разностью двух карточек: она зависит от базовой реализации. Может измениться конфигурация, формат данных и пайплайн контента или архитектурный слой. Поэтому универсальная «стоимость переписывания» не рассчитывается; для конкретной пары оставляется краткая карточка позднего внедрения и поле предметного анализа.

2.6 Модель нагрузки и аппаратной оценки

Вычислительная модель разделяет активную сцену и объём мира. Активная сцена определяет работу на кадр, а масштаб мира дополнительно влияет на потоковую загрузку, резидентную память и общий объём контента. Формулы бюджета кадра и распределения частот приведены в разделе 5. Оценка оборудования остаётся ориентировочной и одновременно проверяет производительность, память и обязательные аппаратные возможности.

2.7 Выводы по разделу 2

Модель связывает входы профиля, каталог решений, фильтрацию, ранжирование и корзину в единую последовательность. Основное методическое требование - не превращать экспертный коэффициент или совпадение двух записей в доказанную причинную связь. Там, где данных недостаточно, система должна показывать ограничение и направлять пользователя к последующей проверке в движке.

3 АРХИТЕКТУРА И РЕАЛИЗАЦИЯ СИСТЕМЫ

3.1 Общая архитектура

Проект реализован как локальное веб-приложение. Пользовательский интерфейс на React и TypeScript отправляет запросы к FastAPI. Backend валидирует профиль, обращается к репозиториям каталога и передаёт данные вычислительным сервисам. Результат возвращается в едином контракте и отображается в интерфейсе рекомендаций, нагрузки, оборудования и корзины.

Таблица 3.1 - Компоненты архитектуры

Компонент	Технологии / файлы	Ответственность
Frontend	React, TypeScript, Vite	анкета, экраны, корзина, сохранение текущего состояния
API	FastAPI, Pydantic	валидация запросов, маршруты каталога и расчёта
Доменное ядро	rules.py, recommender.py	применимость, риски, ранжирование и объяснения
Расчёт ресурсов	hardware.py	стоимость кадра, память, оборудование и ограничения
Данные	SQLAlchemy, SQLite, Alembic	каталог, связи, источники, миграции и целостность
Наполнение	seed/*.py, seed/data	первичное наполнение опубликованного каталога
Контроль качества	pytest, Vitest, CI	модульные, интеграционные и сборочные проверки

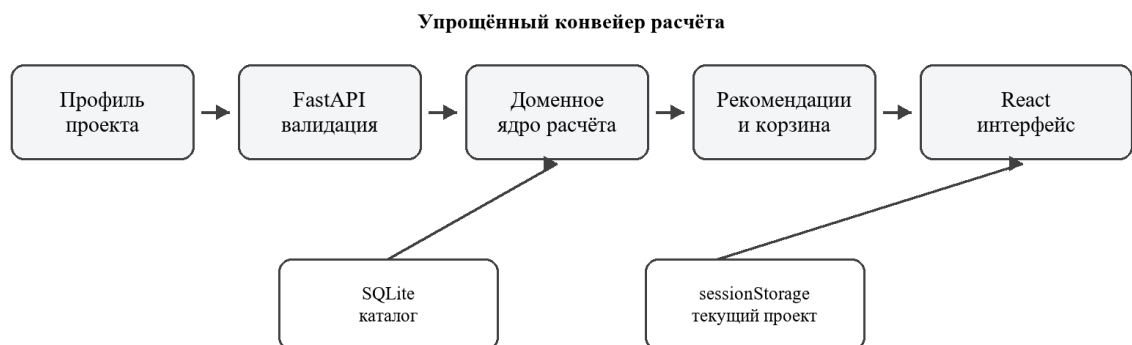


Рисунок 3.1 - Упрощённый конвейер формирования результата

В собранном frontend запросы к адресу /api проксируются на backend по адресу 127.0.0.1:8000. В production-сборке FastAPI может раздавать

собранные статические файлы frontend. База данных по умолчанию хранится локальным SQLite-файлом; отдельная установка сервера базы данных не требуется.

3.2 Предметные данные и публикация

Основные сущности базы данных - игровая функция, метод, движок, инструмент движка, связь метода с инструментом, связь между методами, аппаратная запись CPU/GPU и журнал проверки. Для публичного расчёта используются опубликованные записи с указанным источником. Состояние публикации отделено от исследовательских черновиков.

Таблица 3.2 - Текущее наполнение локального каталога

Объект	Количество	Статус / оговорка
Игровые функции	36	текущий каталог функций
Методы	111	все имеют статус published и источник в текущей базе; это не заменяет проверку содержания
Движки / инструменты	7 / 70	связь с конкретной технологией показывается отдельно
Связи метод - инструмент	473	тип связи может быть прямым, частичным, альтернативным или диагностическим
Связи между методами	20	конфликты, зависимости и дополнения требуют предметного толкования
CPU / GPU	51 / 79	каталог референсных аппаратных записей
Ошибки целостности	0	текущее чтение validation_issues из локальной базы

Из 111 методов 48 записей в текущей базе ссылаются на Wikipedia. Это не делает запись недействительной автоматически, но означает, что статус published следует понимать как статус наполнения каталога, а не как независимое доказательство каждой причинной связи. Четыре записи не имеют прямого численного эффекта по ресурсам; для них требуется либо объяснение области эффекта, либо уточнение данных.

3.3 API и единый результат расчёта

Таблица 3.3 - Основные API-операции

Маршрут	Назначение	Результат
GET /api/health	проверка готовности	версия и состояние базы
POST /api/recommend	рекомендации	методы, ранги, причины, риски и корзина
POST /api/load-profile	сводная нагрузка корзины	CPU/GPU/RAM/VRAM и качественные эффекты
POST /api/hardware-estimate	оборудование	референсные CPU/GPU, память и оговорки
GET /api/catalog/methods	каталог методов	паспорта опубликованных решений
GET /api/catalog/engines	каталог движков	версии и инструменты
GET /api/docs	интерактивная документация	схема API Swagger

Единый ответ рекомендаций включает отпечаток входа, версию алгоритма и версию каталога. Отпечаток строится по нормализованным данным профиля и корзины, поэтому результат можно проверить на соответствие текущему состоянию. Внутри ответа находятся также профиль нагрузки, аппаратная оценка и список применённых или исключённых методов.

3.4 Состояние frontend и сохранение текущего проекта

Frontend хранит одну текущую анкету и одну текущую корзину в sessionStorage под ключом gamedev_dss_project_v1. История версий намеренно не включена в границы текущей версии: сравнение выполняется через изменение текущего набора и повторный расчёт. При изменении входов результат инвалидируется. Для сетевого запроса создаётся AbortController, а ответ проверяется по отпечатку запрошенного состояния.

Такое решение соответствует задаче поддержки одного текущего черновика. Если в дальнейшем понадобится история архитектурных вариантов, её следует проектировать отдельно, чтобы не смешать журнал изменений с источником текущего расчёта.

3.5 Запуск на Windows и Linux

Текущий автоматический сценарий запуска оформлен для Windows через `start.bat` и отдельные `run.bat`. Он создаёт виртуальное окружение, устанавливает зависимости, запускает `backend` и `frontend`, проверяет `/api/health` и наличие React-контейнера, после чего открывает браузер. Полный сценарий быстрого старта требует финальной проверки после очистки проекта; место для результата оставлено в приложении В.

Код приложения рассчитан также на запуск на Linux. FastAPI, SQLAlchemy, SQLite, React, TypeScript и Vite не требуют Windows API. На Linux используются каталоги `.venv/bin` вместо `.venv\Scripts`, а текущие batch-файлы заменяются shell-скриптом либо ручным запуском двух сервисов. CI использует `ubuntu-latest` и выполняет тесты, миграции, проверку типов и production-сборку, что подтверждает переносимость цепочки сборки. Отдельную проверку запуска на реальной Linux-системе следует добавить после подготовки `start.sh`.

3.6 Выводы по разделу 3

Архитектура разделяет интерфейс, API, вычислительное ядро, каталог и расчёт ресурсов. Сохранение текущего состояния и отпечаток результата защищают от показа ответа для старого профиля. Основной проект завершён по функциональной структуре; последующие действия относятся к финальной упаковке, проверке запуска и сокращению тестового набора.

4 ТЕХНИЧЕСКИЕ МЕТОДЫ И ИХ ВЛИЯНИЕ НА ИГРУ

4.1 Механизмы, эффекты и ограничения методов

Технический метод рассматривается через заменяемую работу, механизм, новые расходы и способ проверки. Нельзя считать оптимизацию безусловной скидкой: ускорение одной стадии может добавить вычислительный проход, память, время сборки, задержку или ограничения качества. Например, рост ossurance помогает скрывать задержки, но сам по себе не гарантирует ускорение и может ухудшить поведение кэша [2].

В каталоге выделены семейства culling; LOD/HLOD и impostors; batching и instancing; streaming, virtual texturing и compression; caching и pooling; baking и precompute; GPU offload и meshlets; upscaling, temporal и frame generation; tick/update budgeting. Для каждого семейства карточка должна фиксировать сокращаемую работу, новые затраты, условия и проверку. Формулировки описывают механизм и не задают универсальный процент ускорения.

4.1.1 Управление видимостью и детализацией

Culling исключает из дальнейшей обработки объекты, которые не должны влиять на текущий вид. LOD выбирает представление с меньшей детализацией по расстоянию или экранной ошибке, а HLOD объединяет элементы иерархии. Импостор заменяет сложную геометрию подготовленным представлением. Все варианты требуют проверки видимости, расстояний перехода, теней, силуэта и стоимости подготовки структур. В небольшой сцене стоимость самого механизма может оказаться сопоставимой с выигрышем, поэтому решение должно подтверждаться измерением конкретной сцены.

4.1.2 Batching и инстансинг

Batching объединяет объекты или команды с совместимыми материалами и состояниями, а instancing использует одну геометрию для многих экземпляров. Ограничения материалов, шейдеров и видимости определяют, действительно ли уменьшается работа. Документация Unity отдельно указывает, что dynamic batching переносит подготовку на CPU и не всегда выгоден [3]. Документация Godot также подчёркивает, что объединённая геометрия может потерять отдельное отсечение [4]. Поэтому в карточке нужно фиксировать не просто «меньше draw calls», а условия, при которых это достигается.

4.1.3 Потокковая загрузка, виртуальное текстурирование и сжатие

Потокковая загрузка поддерживает в памяти только часть мира и подготавливает следующую часть по запросу. Виртуальное текстурирование работает со страницами или плитками, а кэш выбирает резидентные данные. Управление residency и загрузочными ресурсами является отдельной задачей: документация Direct3D 12 разделяет резидентность ресурсов [5] и временные ресурсы загрузки [6]. Поэтому в модели разделены резидентный объём, транзитный пул, декомпрессия и возможный пик памяти.

Блочное сжатие текстур уменьшает полезную нагрузку, но итоговый размер зависит от формата, выравнивания, размещения и временных копий. Для BC-форматов исходным основанием служит описание блоков 4x4 [7]. Сжатие нельзя без дополнительных данных превращать в фиксированную экономию RAM, VRAM и CPU одновременно.

4.1.4 Предвычисление, кэширование и расписание

Baking переносит часть вычислений из времени исполнения в этап подготовки контента. Это может уменьшить runtime-работу, но добавляет данные и ограничивает изменения сцены. Кэширование и pooling уменьшают повторные аллокации или вычисления, однако удерживают объекты и требуют корректной инвалидации. Снижение частоты AI или физики должно рассматриваться отдельно от частоты вывода кадра: в движках фиксированные обновления могут накапливаться и выполняться несколькими шагами [8].

Если оптимизация переносит работу на GPU, требуется учитывать подготовку команд, память и синхронизацию. Нельзя делать вывод о выгоде только по тому, что операция выполняется параллельно. Для будущего измерительного протокола полезны захват времени кадра и инструментированное профилирование; в проекте PresentMon [9] и Trasy рассматриваются как внешние кандидаты, а не встроенные зависимости.

4.1.5 Трассировка лучей, материалы и постобработка

Трассировка лучей состоит как минимум из подготовки структур ускорения и traversal. В руководстве Unreal Engine различаются обновления BLAS и построение TLAS, а затраты связываются с потоками рендера и GPU [10]. Поэтому RT не должен учитываться как одна универсальная скидка или фиксированное число миллисекунд для любой игры.

Апскейлинг снижает внутреннее разрешение части пиксельных стадий и добавляет собственный проход восстановления. Генерация кадров имеет отдельную стоимость синтеза и не должна уменьшать стоимость уже отрисованных кадров. Для материалов важны конкретные вычисления шейдера и приближения: документация Filament показывает, что даже отдельные параметры PBR имеют собственную обработку и ограничения [11].

Аппаратно-зависимое сжатие может выполняться не тем устройством, которое ожидает разработчик. Например, DirectStorage допускает CPU fallback для декомпрессии [12]. Поэтому в паспорте следует указывать ресурс исполнения и не переносить свойство Windows API на Linux без отдельной проверки.

4.2 Геймплейные решения как технические требования

Таблица 4.1 - Связь геймплейного требования и архитектуры

Геймплейное решение	Технические последствия	Ограничение вывода
Разрушаемость окружения	динамическая геометрия, физические тела, обновление коллизий, сохранение состояния, возможная репликация	сам факт разрушаемости не задаёт число объектов и частоту событий
Большой мир	стриминг уровней и страниц, HLOD, память, размер контента, I/O	масштаб мира не равен числу одновременно активных сущностей
Большое количество NPC	AI, поиск пути, анимация, perception, LOD, расписание обновлений	количество NPC не определяет сложность поведения без модели задач
Мультиплеер	авторитет, репликация, сериализация, топология, bandwidth, детерминизм	число игроков не равно сетевой нагрузке без частоты и объёма состояния
Высокое разрешение и FPS	бюджет кадра и пиксельные стадии рендера	FPS не является гарантией, если нет трассы конкретной игры

Разделение активной сцены и объёма мира особенно важно для открытого мира. При одинаковом количестве активных объектов размер мира может увеличить резидентную память и стоимость потоковой загрузки, но не обязан увеличивать стоимость каждого кадра. Это правило используется и в текущей реализации расчёта.

4.3 Интеграция, совместимость и стоимость замены

Методы влияют друг на друга через общие данные и этапы пайплайна: LOD может конфликтовать с ручным объединением геометрии, virtual texturing связано с residency и streaming pool, baking ограничивает динамику, а frame generation требует отдельного анализа базовой и отображаемой частоты. Общая цель или общий ресурс ещё не доказывают конфликт.

В карточке разделяются первоначальное и позднее внедрение, совместимость и фактическая стоимость перехода. Последняя зависит от базовой архитектуры и без анализа конкретной пары не рассчитывается: изменение параметра означает настройку, формата данных - переработку контента, архитектурного слоя - возможную существенную переработку. При нехватке сведений используется «требует анализа» или пустое поле. Исключённые решения уже определены границами предметной области в разделе 1; универсальные утверждения об ускорении не используются.

5 МАТЕМАТИЧЕСКИЕ МОДЕЛИ, ПРОВЕРКА И РЕЗУЛЬТАТЫ

5.1 Модель ресурсной нагрузки

Вычислительная модель разделяет активную сцену и объём мира. Активная сцена определяет работу на кадр, а масштаб мира дополнительно влияет на потоковую загрузку, резидентную память и общий объём контента. Такое разделение предотвращает повторное умножение стоимости кадра только из-за размера мира.

$$V_{\text{кадр}} = 1000 / f_{\text{рендер}} \quad (5.1)$$

Бюджет кадра определяется частотой реально отрисованных кадров. Физика и AI имеют собственные частоты обновления. Их вклад распределяется по кадрам по правилу:

$$L_{\text{кадр}} = L_{\text{такт}} \times f_{\text{такт}} / f_{\text{рендер}} \quad (5.2)$$

Для CPU отдельно учитываются последовательный критический путь и условно распараллеливаемая часть. Для GPU разделяются растеризация, пиксельные стадии, вычисления и трассировка лучей. RAM и VRAM представляются как сумма поименованных компонентов, а повторяющаяся экономия одного и того же компонента не должна учитываться дважды.

Оценка оборудования выбирает минимальный класс из каталога, одновременно проверяя вычислительные индексы, объём памяти и обязательные аппаратные возможности. Это расчётная оценка, а не измерение конкретной игры.

5.2 Ранжирование методом TOPSIS

TOPSIS применяется к матрице «альтернатива - критерий». Критерии делятся на выгодные, где большее значение предпочтительно, и затратные, где предпочтительно меньшее значение. После нормализации строки умножаются на веса, зависящие от приоритета пользователя.

$$r_{ij} = x_{ij} / \sqrt{(\sum_i x_{ij}^2)} \quad (5.3)$$

Для каждого критерия строятся положительное и отрицательное идеальные решения. Коэффициент близости альтернативы к идеальному решению вычисляется через евклидовы расстояния:

$$C_i = d_i^- / (d_i^+ + d_i^-) \quad (5.4)$$

Чем выше C_i , тем ближе вариант к положительному идеалу. При одной альтернативе или одинаковых строках матрицы строгий порядок не имеет смысла. Разница менее 0,02 трактуется как группа практически равнозначных вариантов, а не как доказанное превосходство лидера. Для проверки устойчивости веса критериев по одному изменяются на $\pm 10\%$.

Расчётный пример с условными значениями не является измерением производительности: он нужен только для проверки направления формулы. Фактические значения текущей модели приведены в таблице чувствительности ниже.

5.3 Организация проверки

Проверка системы состоит из трёх уровней. Модульные тесты проверяют отдельные правила и формулы. Интеграционные тесты отправляют реальные запросы к FastAPI и проверяют связность каталога, расчёта и базы. Frontend-тесты проверяют состояние интерфейса, сохранение, инвалидирование результата и обработку устаревших ответов. Сквозной smoke-check обращается к запущенному сервису и подтверждает работу основных пользовательских сценариев.

В текущем проекте накоплен расширенный набор проверок. Для учебного отчёта не требуется перечислять каждую проверку: основной текст фиксирует свойства, которые должны быть подтверждены приёмочным набором, а полный набор сохраняется как регрессионный материал.

5.4 Чувствительность к изменению параметров профиля

В качестве базового сценария использован 3D-проект с открытым миром, высоким качеством, разрешением 1440p, целевой частотой 60 FPS и выбранными функциями потоковой загрузки и толпы NPC. Значения индексов являются внутренними относительными величинами текущей модели, а RAM/VRAM и draw calls - расчётными оценками.

Таблица 5.1 - Изменение результата при изменении параметров профиля

Сценарий	CPU index	GPU index	RAM, ГБ	VRAM, ГБ	Draw calls
База: 60 FPS, 1440p	0.3057	0.3216	14.9	10.9	40535
30 FPS	0.1529	0.1608	14.9	10.9	40535
144 FPS	0.7337	0.7719	14.9	10.9	40535
720p	0.3057	0.1608	14.4	7.7	40535
2160p	0.3057	0.5513	15.7	15.5	40535
Масштаб small	0.3057	0.3216	12.3	8.4	27419
Масштаб very_large	0.3057	0.3216	16.4	12.3	47822

Результат соответствует ожидаемому направлению модели. Повышение целевой частоты уменьшает бюджет одного кадра и повышает относительные индексы требований. Повышение разрешения влияет прежде всего на GPU и целевые буферы. Изменение масштаба мира при неизменной активной сцене увеличивает память и объём контента, но не изменяет CPU/GPU-стоимость кадра. Это различие является принципиальным: иначе размер мира ошибочно считался бы повторной работой на каждом кадре.

Для финальной версии следует повторить таблицу после изменения каталога или коэффициентов и заменить значения в отмеченном месте, если они изменятся.

5.5 Изменение и сохранение корзины

Для базового профиля было проверено добавление метода «Временное масштабирование изображения». При пустой корзине выбранных методов нет. После добавления метод появляется в `selected_methods`, отпечаток входа изменяется, а сводный профиль нагрузки получает отдельные эффекты метода.

Таблица 5.2 - Пример реакции на изменение корзины

Состояние	Изменение результата	Интерпретация
Пустая корзина	selected_methods пуст; input_key соответствует профилю	расчёт только по профилю и общим правилам
Добавлено temporal_upscaling	GPU raw -0.0272; RAM +0.0067; VRAM +0.0275	снижается часть пиксельной стоимости, но добавляются буферы и проход восстановления
Изменён порядок кодов	отпечаток не меняется	набор, а не порядок выбора является входом
Метод заменён другим	selected_methods и input_key меняются	результат строится для нового набора; состав рекомендаций может остаться тем же

Изменение корзины не обязано менять первые места TOPSIS. В сценарии изменились selected_methods, отпечаток и сводная нагрузка, а топ-5 остался тем же: корзина описывает выбранные решения, а кандидаты ранжируются отдельно.

Корзина хранится в черновике браузера и после обновления восстанавливает профиль и набор. Изменение профиля или корзины сбрасывает предыдущий ответ, а поздний ответ старого запроса отбрасывается. Трудоемкость замены зависит от конкретной пары методов и оставляется для предметного анализа.

5.6 Ограничения достоверности и аппаратной оценки

Проверки подтверждают, что ограничения RAM и VRAM обрабатываются как жёсткие условия: при слишком малом пределе система сообщает о нарушении. Если расчёт превышает доступный каталог, вместо случайного выбора возвращается признак exceeds_catalog. Аппаратные записи проверяются одновременно по производительности, памяти и обязательным возможностям, включая поддержку трассировки лучей.

Оценка не содержит поля гарантированного FPS и сопровождается оговорками об экспертных коэффициентах, неизвестных входах и необходимости измерения на конкретной сцене. Это ограничение является частью корректного результата, а не отказом от расчёта.

5.7 Текущее состояние тестов и сборки

Таблица 5.3 - Результаты контрольного прогона текущей версии

Область	Результат	Статус для отчёта
Backend pytest	452 passed, 2 skipped	текущая расширенная регрессия; обновить после финального сокращения
Frontend Vitest	34 passed	текущая проверка store и компонентов; обновить после чистки
TypeScript	без ошибок	проверка типов пройдена
Production build	сборка завершена	проверка frontend пройдена
Smoke-check	85/85 по текущему сценарию	повторить после финальной проверки быстрого старта
CI	Ubuntu: тесты, миграции, типы, сборка	подтверждает воспроизводимость цепочки

Для учебного проекта проверки разделяются на основной приёмочный набор и расширенную регрессию. Основной набор подтверждает запуск, изменение параметра профиля, RAM/VRAM-ограничения, корзину и сохранение, одну связь решений, отсутствие обещания FPS и сборку; остальные проверки остаются регрессионным материалом.

[После очистки указать финальное количество приёмочных и расширенных тестов.]

В проекте нет независимой трассы CPU/GPU, измеренного FPS, пиков памяти и стенда для подтверждения универсальных коэффициентов. Поэтому числа называются расчётными индексами и ориентировочными оценками. Поля `source_url` и `published` подтверждают заполненность карточки, но не индивидуальную проверку каждой связи и влияния; стоимость перехода без описания базовой реализации остаётся пустой или условной.

Проект работает локально, не содержит встроенного runtime-профилировщика и не применяет решения автоматически. На Windows используется batch-сценарий; Linux требует отдельного shell-скрипта.

Итог раздела 5: расчёт реагирует на значимые параметры профиля, корзина меняет выбранный набор и сводный результат, а устаревшие ответы не должны подменять текущее состояние. После чистки тестов обновляются количественные показатели и ссылки на финальные сценарии.

ЗАКЛЮЧЕНИЕ

В рамках проекта разработана локальная информационная система поддержки принятия решений по технической оптимизации разработки компьютерных игр. Система связывает профиль проекта, игровые функции, методы реализации, инструменты движков, ограничения и ориентировочную аппаратную оценку. Основная функциональность проекта реализована и объединена в единый пользовательский сценарий.

В отчёте сформулирована граница технического анализа. В неё входят решения, влияющие на архитектуру, вычисления и ресурсы: разрушаемость, масштаб мира, количество NPC, мультиплеер, рендеринг, память, потоковая загрузка, физика, AI и сетевые подсистемы. Монетизация, дата выхода, сюжетные развилки и количество концовок исключены из расчёта, поскольку не задают в данной модели проверяемую техническую нагрузку.

Реализованный расчёт использует фильтрацию применимости, TOPSIS, анализ устойчивости, единый профиль нагрузки и подбор референсного класса оборудования. Показано, что целевой FPS и разрешение меняют соответствующие индексы, а размер мира при одинаковой активной сцене влияет прежде всего на память и потоковую загрузку. Оценка оборудования остаётся ориентировочной и не является гарантией FPS.

Проверка корзины показала, что изменение состава набора меняет отпечаток входа, список выбранных методов и сводное влияние ресурсов. При этом рейтинг свободных кандидатов может не измениться, что не является ошибкой. Сложность перехода от уже реализованного решения к другому зависит от базовой архитектуры, поэтому она не заполняется универсальным числом без дополнительного анализа.

Основные оставшиеся действия относятся к финальной редакции: проверить формулировки карточек и источников, удалить неподтверждённые причинные выводы, завершить анализ взаимодействий методов, сократить основной набор тестов, проверить быстрый старт на Windows и Linux и обновить количественные показатели в отмеченных местах. Эти действия не меняют того, что основная разработка системы завершена.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] GAMORRA: A GPU and CPU Performance Modeling Framework for Games (arXiv 2204.11025, 2022). <https://arxiv.org/pdf/2204.11025> использовано для разделения параметров, модели и измерительного протокола
- [2] Guthmann F. Occupancy explained. AMD GPUOpen, 2023; обновление 2024. <https://gpuopen.com/learn/occupancy-explained/> использовано для ограничения вывода об occupancy
- [3] Draw call batching. Unity Manual 6000.0. <https://docs.unity3d.com/6000.0/Documentation/Manual/DrawCallBatching.html> использовано для условий batching
- [4] GPU optimization. Godot Engine documentation 4.4. https://docs.godotengine.org/en/4.4/tutorials/performance/gpu_optimization.html использовано для batching, материалов и pixel cost
- [5] D3D12 Residency. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/direct3d12/residency> использовано для различения резидентности и рабочего набора
- [6] Uploading resources. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/direct3d12/uploading-resources> использовано для описания временных ресурсов загрузки
- [7] Texture block compression in Direct3D 11. Microsoft Learn. <https://learn.microsoft.com/en-us/windows/win32/direct3d11/texture-block-compression-in-direct3d-11> использовано для ограничения расчёта размера текстур
- [8] Fixed updates. Unity Manual 6000.0. <https://docs.unity3d.com/6000.0/Documentation/Manual/fixed-updates.html> использовано для разделения частоты такта и частоты кадра
- [9] PresentMon. Intel GameTechDev. <https://github.com/GameTechDev/PresentMon> рассматривается как внешний инструмент измерения времени кадра
- [10] Ray Tracing Performance Guide in Unreal Engine 5.5. Epic Games. https://dev.epicgames.com/documentation/en-us/unreal-engine/ray-tracing-performance-guide-in-unreal-engine?application_version=5.5 использовано для разделения BLAS/TLAS и RT-проходов
- [11] Filament: Physically Based Rendering. Google. <https://google.github.io/filament/Filament.md.html> использовано для конкретизации обработки материалов и приближений
- [12] DirectStorage compression support. Microsoft Learn. https://learn.microsoft.com/en-us/windows/win32/dstorage/dstorage/ne-dstorage-dstorage_compression_support использовано для ограничения вывода о декомпрессии

ПРИЛОЖЕНИЕ А. СВОДКА ТЕКУЩЕГО СОСТОЯНИЯ ПРОЕКТА

Приложение предназначено для обновления после финальной очистки. В основной текст отчёта следует переносить только те числа, которые подтверждены текущим прогоном и совпадают с каталогом, используемым приложением.

Таблица А.1 - Состояние проекта на момент подготовки рабочей версии

Область	Текущее состояние	Место обновления
Backend	FastAPI + SQLAlchemy + SQLite; расчёт, каталог, миграции	[версия / commit]
Frontend	React + TypeScript + Vite; 11 экранов по текущему README	[уточнить итоговое число экранов]
Методы	111 published, все с source_url	[проверить спорные карточки]
Игровые функции	36	[обновить при изменении seed]
Движки / инструменты	7 / 70	[обновить при изменении seed]
Аппаратный каталог	51 CPU / 79 GPU	[обновить при изменении seed]
Тесты	452 backend; 34 frontend до финальной чистки	[вписать основной и расширенный набор]
Быстрый старт	Windows batch; Linux ручной запуск / будущий shell-скрипт	[вписать результат финального прогона]

ПРИЛОЖЕНИЕ Б. КОМПАКТНЫЙ ПРИЁМОЧНЫЙ ПРОТОКОЛ

Протокол предназначен для основного учебного прогона после сокращения расширенной регрессии. Каждый сценарий должен использовать чистую временную базу и фиксированный профиль.

Таблица Б.1 - Минимальный набор приёмочных проверок

№	Проверка	Ожидаемый результат	Факт
1	Запуск и health-check	backend отвечает, frontend открывается, база доступна	[]
2	Один параметр профиля	при изменении FPS или разрешения меняются соответствующие индексы	[]
3	Разделение мира и активной сцены	масштаб меняет память/контент, но не кадр при той же активной сцене	[]
4	Аппаратные ограничения	нарушение RAM/VRAM и отсутствие подходящего класса сообщаются явно	[]
5	Корзина	добавление/замена меняет selected_methods, input_key и сводный профиль	[]
6	Сохранение	после обновления восстанавливаются профиль и корзина; старый ответ сбрасывается	[]
7	Совместимость	конфликт или зависимость показывается с объяснением	[]
8	Сборка	typecheck и production build завершаются без ошибок	[]

Расширенные тесты, проверки безопасности, калибровочные прогоны и исторические контрпримеры следует оставить в проекте только при наличии практической пользы. В основной отчёт достаточно включить их классификацию и итоговый статус.

ПРИЛОЖЕНИЕ В. ПОЛЯ ДЛЯ ФИНАЛЬНОГО ОБНОВЛЕНИЯ

Таблица В.1 - Изменения, которые можно внести без перестройки отчёта

Поле	Что добавить	Готово
Тесты	итоговое число приёмочных и расширенных тестов после чистки	[]
Быстрый старт Windows	дата, команда и результат запуска	[]
Быстрый старт Linux	результат запуска через start.sh или ручные команды	[]
Каталог методов	проверенные карточки и ссылки первичных источников	[]
Взаимодействия	условия для конфликтов, зависимостей и complement	[]
Корзина	конкретный пример замены решения и трудоёмкости перехода	[]
Профиль	финальные значения таблицы чувствительности	[]
Титульный лист	организация, кафедра, ФИО, группа, руководитель, город	[]
Использование ИИ	указать факт и роль инструмента по требованиям преподавателя; итоговый текст проверить и изложить автором	[]

[Поля в квадратных скобках предназначены для ручного обновления автором после финального прогона.]